THE COMPLETE GUIDE

Agentic RAG

Letting the LLM Decide When & What to Retrieve

Move beyond fixed retrieval pipelines. Build dynamic systems that route, grade, and self-correct.

POWERED BY PRODUCTION STACK

LangGraph | HuggingFace | Open-Source Models


AI Coach John | PROITBRIDGE Strive For Better Future

PARADIGM SHIFT

Traditional RAG vs Agentic RAG

What is actually different here?

TRADITIONAL RAG

Fixed pipeline. Every query proceeds sequentially:
Retrieve → Generate.

No matter what the question is, it always hits the vector store exactly once.

AGENTIC RAG

LLM inside the loop. It acts as an active decision-maker that decides:

- *Whether* to retrieve
- *What* to search for
- *How many times* & *which source*
- *When* it has enough context

THE CORE SHIFT

Retrieval goes from being a hardcoded pipeline step to being one of several actions an LLM can choose to take - just like calling a calculator or a web search tool.

PIPELINE VS LOOPING AGENT

USE CASES

Why Go Agentic?

When agentic RAG earns its complexity

MULTI-SOURCE ASSISTANTS

Deciding between internal docs, web search, or a SQL database depending on the specific question.

MULTI-HOP QUESTIONS

"Compare Q1 and Q3 revenue growth" needs two separate retrievals and reasoning across both data points.

MIXED QUERY TYPES

A support bot seamlessly handling both factual questions (needs retrieval) and casual chit-chat (doesn't).

SELF-CORRECTING RESEARCH

Systems where a bad or irrelevant first retrieval needs to be actively noticed, graded, and retried.

When you DO NOT need it:

Consistent, single-hop queries over one source - a standard RAG pipeline is simpler, cheaper, and more predictable. Do not add complexity you do not need!

ARCHITECTURE LANDSCAPE

Named Agentic RAG Patterns

The four core patterns seen in research papers & frameworks

SELF-RAG

Model generates special reflection tokens that critique its own retrieval and output as it goes, deciding mid-generation if it needs more data.

CORRECTIVE RAG (CRAG)

Grades retrieved docs; if they are weak or irrelevant, falls back to a broader source (like web search) instead of answering badly.

ADAPTIVE RAG

Routes queries by complexity: simple queries are answered directly, while complex ones go through multi-step retrieval workflows.

REACT-STYLE RAG

Classic reason - act - observe loop, where "act" is calling the retrieval tool and "observe" is reading the result before the next move.

WHAT WE WILL BUILD:
A pipeline borrowing the best ideas from all four - routing, grading, correction, and a reasoning loop - implemented as one LangGraph graph.

ARCHITECTURE

Core Building Blocks

What an agentic RAG system is made of

1 Retrieval as a Tool
Exposed to LLM as a callable function

2 Decision / Router Node
Decides whether/what/where to search

3 Grading Node
Checks if retrieved docs are useful

4 Correction Path
Fallback when local retrieval fails

5 The Loop
Cycles through search before generating

6 Memory
Tracks state across loops & turns

>_ REQUIRED DEPENDENCIES

```
pip install langgraph langchain langchain-community \
            sentence-transformers faiss-cpu transformers \
            duckduckgo-search rank-bm25
```

AI Coach John | PROITBRIDGE Strive For Better Future | Follow | Repost

STEP 1

Expose Retrieval as a Tool

Wraps the retriever in a function with a clear description so the LLM can call it

```
from langchain.tools import tool

@tool
def retrieve_docs(query: str) → str:
    """Search the internal knowledge base for information about
company
    policies, products, or documentation. Use this for factual
questions
    that require grounding in internal documents."""
    results = retriever.get_relevant_documents(query)
    return "\n\n".join([doc.page_content for doc in results[:3]])
```

KEY IMPLEMENTATION NOTES

- **Docstring is critical:** The docstring is what the LLM reads to decide *whether* this tool applies - vague descriptions cause over- or under-calling.
- **Optional retrieval:** This is the key shift from standard RAG: retrieval becomes optional, not forced by a rigid pipeline.
- **One tool per source:** You will define one tool like this per data source (docs, web, SQL database) - covered in Slide 15.

STEP 2

The Decision / Router Node

Before retrieving, the LLM decides if external knowledge is actually needed

```
from langchain_core.prompts import ChatPromptTemplate

decision_prompt = ChatPromptTemplate.from_template("""
Decide if answering this question requires searching the knowledge
base.
Answer "RETRIEVE" if it needs factual/company-specific
information.
Answer "DIRECT" if you can answer confidently without lookup.

Question: {question}
Decision:""")

def decide_node(state):
    decision =
llm.invoke(decision_prompt.format(question=state["question"]))
    state["route"] = "retrieve" if "RETRIEVE" in decision.content
else "direct"
    return state
```

WHY THIS MATTERS

- **Narrow & binary job:** This node's only job is to output a decision, not an answer - keep it narrow and binary for maximum reliability.

- **Production logging:** Log these decisions in production to catch over-retrieving (wasteful cost) or under-retrieving (hallucination risk).

STEP 3

Query Reformulation

Letting the LLM decide *what* to search for by rewriting raw input

```
rewrite_prompt = ChatPromptTemplate.from_template("""
Rewrite this question into a clear, specific search query
optimized
for retrieval. Resolve pronouns and vague references using the
conversation history if needed. Return only the query.

History: {history}
Question: {question}
Search query:""")

def rewrite_node(state):
    query = llm.invoke(rewrite_prompt.format(
        history=state.get("history", ""),
question=state["question"]
    ))
    state["search_query"] = query.content.strip()
    return state
```

WHY REWRITE?

- **Fix conversational input:** Raw conversational questions ("what about last year's?") are bad vector-search input as-is.
- **Resolve ambiguity:** This step resolves ambiguity and folds in conversation history - same underlying idea as HyDE (Hypothetical Document Embeddings), just simpler and faster.

STEP 4

Query Decomposition (Multi-Hop)

Breaking complex questions into simpler sub-questions

```
decompose_prompt = ChatPromptTemplate.from_template("""
If this question requires multiple pieces of information to
answer,
break it into a numbered list of simpler sub-questions. If it's
already
simple, return it as a single item.

Question: {question}
Sub-questions:""")

def decompose_node(state):
    output =
llm.invoke(decompose_prompt.format(question=state["question"]))
    sub_questions = [q.strip("- 0123456789.") for q in
output.content.split("\n") if q.strip()]
    state["sub_questions"] = sub_questions
    return state
```

ENABLING MULTI-HOP REASONING

Example: "Compare Q1 and Q3 revenue" becomes two sub-questions: "What was Q1 revenue?" and "What was Q3 revenue?" - each retrieved separately, then reasoned over together.

Avoid weak retrieval: This makes true multi-hop reasoning possible - a single retrieval call over a combined question usually returns weak,

STEP 5

Hybrid Retrieval Execution

Combining dense vector similarity with sparse keyword search (BM25)

```
from rank_bm25 import BM25Okapi

def hybrid_retrieve(query, faiss_index, bm25_index, chunks,
top_k=5, alpha=0.5):
    dense_results = retrieve(query, faiss_index, chunks,
top_k=top_k)
    bm25_scores = bm25_index.get_scores(query.split())
    sparse_top = sorted(
        zip(bm25_scores, chunks), key=lambda x: x[0], reverse=True
    )[:top_k]

    combined = {c.page_content: c for c in dense_results}
    for _, c in sparse_top:
        combined[c.page_content] = c
    return list(combined.values())
```

DENSE + SPARSE SYNERGY

Best of both worlds: Dense (embeddings) is great at semantic meaning; sparse (BM25) is great at exact keyword/code/ID matches - combining both covers more query types.

Alpha weighting: `alpha` weights one method over the other depending on domain (e.g., legal/technical docs benefit from more keyword weight).

STEP 6

Grading Retrieved Documents

Evaluating if retrieved chunks are relevant before generating an answer

```
grade_prompt = ChatPromptTemplate.from_template("""
Do these documents contain enough information to answer the
question?
Answer only "YES" or "NO".

Question: {question}
Documents: {documents}
Answer:""")

def grade_node(state):
    grade = llm.invoke(grade_prompt.format(
        question=state["question"], documents=state["retrieved"]
    ))
    state["sufficient"] = "YES" in grade.content
    return state
```

CORE OF SELF-CORRECTION

Stop blind trust: Standard RAG blindly trusts whatever it retrieved; agentic RAG explicitly checks relevance first.

Smart fallback routing: If grading fails, do not just retry blindly - route to a correction step with feedback about what was missing.

STEP 7

Corrective Retrieval (Web Fallback)

What to do when local retrieval fails (CRAG pattern)

```python
from duckduckgo_search import DDGS

@tool
def web_search(query: str) → str:
    """Search the web. Use this when internal documents don't have
    enough information to answer the question."""
    with DDGS() as ddgs:
        results = list(ddgs.text(query, max_results=3))
    return "\n\n".join([r["body"] for r in results])

def correct_node(state):
    state["retrieved"] = web_search.invoke(state["search_query"])
    state["source"] = "web"
    return state
```

ESCALATE INSTEAD OF HALLUCINATING

↑ **The CRAG pattern:** Do not just fail silently or hallucinate - escalate to a different, broader source like the web.

Tag source trust levels: Always tag the source (`state["source"]`) so the final answer discloses where information came from - internal docs vs open web carry very different trust levels.

STEP 8

Self-Reflection on Answer

Critiquing the answer before returning it (Self-RAG pattern)

```
reflect_prompt = ChatPromptTemplate.from_template("""
Does the answer below rely only on the given context, with no
invented
facts? Answer "SUPPORTED" or "UNSUPPORTED".

Context: {context}
Answer: {answer}
Verdict:""")

def reflect_node(state):
    verdict = llm.invoke(reflect_prompt.format(
        context=state["retrieved"], answer=state["draft_answer"]
    ))
    state["supported"] = "SUPPORTED" in verdict.content
    return state
```

CATCHING HALLUCINATION EARLY

Critique own output: Self-RAG's core idea is that reflection is not just on retrieved docs, but on the model's own generated output.

Handle unsupported claims: If unsupported, route back to generation with instructions to stick strictly to context, or flag as low-confidence.

STEP 9

Final Answer with Citations

Generating the answer with precise source attribution

```
generate_prompt = ChatPromptTemplate.from_template("""
Answer the question using only the context. Cite which source
number supports each claim, like [1], [2].

Context:
{numbered_context}

Question: {question}
Answer:""")

def generate_node(state):
    numbered = "\n".join([f"[{i+1}] {c}" for i, c in
enumerate(state["retrieved_list"])])
    answer = llm.invoke(generate_prompt.format(
        numbered_context=numbered, question=state["question"]
    ))
    state["answer"] = answer.content
    return state
```

WHY CITATIONS ARE ESSENTIAL

- **Traceable claims:** Numbered citations let users and evaluation pipelines trace every single claim back to a specific chunk.
- **Effortless debugging:** When an answer is wrong, you can immediately see which source pulled the bad claim.

ROUTING

Multi-Tool Routing

Handling more than one retrieval source dynamically

```
tools = [retrieve_docs, sql_query_tool, web_search]
llm_with_tools = llm.bind_tools(tools)

def multi_tool_node(state):
    response = llm_with_tools.invoke(state["question"])
    state["tool_calls"] = response.tool_calls
    return state
```

NATIVE FUNCTION CALLING

Structured tool execution: `bind_tools` gives the LLM native function-calling - it returns structured `tool_calls` instead of freeform text, executed programmatically.

Parallel tool use: The LLM can call more than one tool for a single question (e.g., check SQL for numbers AND docs for policy context) in parallel or sequence.

ARCHITECTURE

Full LangGraph Wiring

Putting all nodes together into a single adaptive state graph

```
from langgraph.graph import StateGraph, END
from typing import TypedDict, List

class AgentState(TypedDict):
    question: str; search_query: str; retrieved: str; retrieved_list: List[str]
    sufficient: bool; supported: bool; retries: int; draft_answer: str; answer:
str; route: str; source: str

graph = StateGraph(AgentState)
graph.add_node("decide", decide_node); graph.add_node("rewrite", rewrite_node)
graph.add_node("retrieve", retrieve_node); graph.add_node("grade", grade_node)
graph.add_node("correct", correct_node); graph.add_node("generate", generate_node)
graph.add_node("reflect", reflect_node)

graph.set_entry_point("decide")
graph.add_conditional_edges("decide", lambda s: s["route"], {"retrieve":
"rewrite", "direct": "generate"})
graph.add_edge("rewrite", "retrieve"); graph.add_edge("retrieve", "grade")
graph.add_conditional_edges("grade", lambda s: "generate" if s["sufficient"] else
("correct" if s["retries"] < 2 else "generate"), {"generate": "generate",
"correct": "correct"})
graph.add_edge("correct", "generate"); graph.add_edge("generate", "reflect")
graph.add_conditional_edges("reflect", lambda s: END if s["supported"] else
"generate", {END: END, "generate": "generate"})
agentic_rag = graph.compile()
```

TWO CONDITIONAL LOOPS

Every branch point uses `add_conditional_edges`. Two separate loops exist: grade to correct (retrieval quality) and reflect to generate (output quality), each capped by retry counters.

SCALING UP

Multi-Agent Agentic RAG

A supervisor routing questions to specialized retriever sub-agents

```
from langgraph.graph import StateGraph

def supervisor_node(state):
    routing_decision = llm.invoke(
        f"Route this question to: docs_agent, sql_agent, or
web_agent.\nQuestion: {state['question']}"
    )
    state["next_agent"] = routing_decision.content.strip()
    return state

# Each sub-agent is itself a compiled LangGraph graph
graph.add_node("supervisor", supervisor_node)
graph.add_conditional_edges(
    "supervisor", lambda s: s["next_agent"],
    {"docs_agent": "docs_agent", "sql_agent": "sql_agent",
"web_agent": "web_agent"}
)
```

THE SUPERVISOR PATTERN

- **Independent domains:** Useful when each domain needs very different retrieval logic, tools, or underlying models - keeps each sub-agent simple and testable.

- **Tradeoff warning:** More moving parts, more LLM calls, more places for errors to hide. Only go multi-agent when a single graph genuinely becomes

STATE MANAGEMENT

Memory: Short vs Long-Term

Handling state across a conversation, not just one query

SHORT-TERM MEMORY

The `AgentState` within a single graph run (question, retrieved docs, retry count). Resets every invocation.

LONG-TERM MEMORY

Conversation history and past interactions, persisted across multiple calls to the graph via checkpointers.

```
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
agentic_rag = graph.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "user-123"}}
result = agentic_rag.invoke({"question": "What's our refund
policy?"}, config=config)
follow_up = agentic_rag.invoke({"question": "What about for
digital products?"}, config=config)
```

PRODUCTION PERSISTENCE

LangGraph's `checkpointer` persists state per `thread_id`. In production, swap `MemorySaver` for a persistent backend (SQLite, Postgres, Redis) so state survives restarts.

AMBIGUITY HANDLING

Handling Ambiguous Queries

Sometimes the right move is to ask the user, not retrieve

```
clarify_prompt = ChatPromptTemplate.from_template("""
Classify this question as one of: RETRIEVE, DIRECT, CLARIFY.
Use CLARIFY only if the question is too vague or ambiguous to act
on.

Question: {question}
Classification:""")

def route_node(state):
    result =
llm.invoke(clarify_prompt.format(question=state["question"]))
    state["route"] = result.content.strip()
    return state
```

WHY ADDING CLARIFY IS CRUCIAL

Prevent blind guessing: Without this, an agent will confidently retrieve and answer even for genuinely ambiguous questions ("tell me about the policy" - which policy?).

Superior UX: A CLARIFY branch routes to a node that returns a follow-up question to the user instead of forcing a guess - much better UX for assistants.

ROBUSTNESS

Error Handling & Fallbacks

What happens when a tool call fails in production?

EMPTY RETRIEVAL

Vector store returns nothing -> route to fallback, do not generate from nothing.

TOOL TIMEOUT

SQL query or web search hangs -> set timeouts and a fallback response.

TOOL ERRORS

Catch exceptions inside tools, return a structured error string, not a stack trace.

INFINITE LOOPS

Always cap retries with a counter in state - never rely on LLM to stop.

```python
@tool
def safe_retrieve(query: str) → str:
    """Search the knowledge base safely."""
    try:
        results = retriever.get_relevant_documents(query)
        if not results:
            return "NO_RESULTS_FOUND"
        return "\n\n".join([doc.page_content for doc in
results[:3]])
    except Exception as e:
        return f"TOOL_ERROR: {str(e)}"
```

OPTIMIZATION

Caching for Cost & Latency

Do not repeat expensive LLM or retrieval work you have already done

Retrieval Caching
Semantic cache using embedding similarity for paraphrased repeat queries.

Decision Caching
If the same question type routes the same way repeatedly, cache the routing decision.

LLM Response Caching
Cache final generated answers for frequently asked questions (FAQs).

```python
from functools import lru_cache

@lru_cache(maxsize=1000)
def cached_retrieve(query: str):
    return retrieve(query, faiss_index, chunks)
```

COMPOUNDING SAVINGS

Caching matters even more in agentic RAG than standard RAG, since a single question triggers several LLM/tool calls. Cache hits compound savings across the entire loop!

USER EXPERIENCE

Streaming Responses

Keeping the UI responsive during a multi-step loop

```
for event in agentic_rag.stream({"question": "Compare Q1 and Q3
revenue"}, config=config):
    for node_name, output in event.items():
        print(f"[{node_name}] → {output}")
```

SURFACE INTERMEDIATE PROGRESS

- **Stream node execution:** LangGraph's `.stream()` yields output after each node completes - surface these as progress indicators ("Searching documents...", "Checking results...") in your UI.

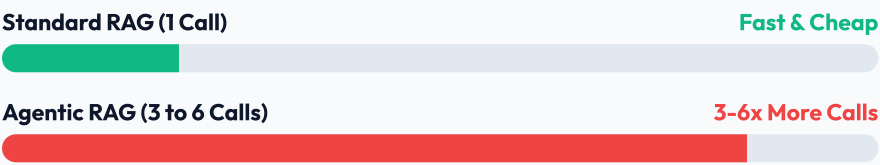
- **Eliminate silent waiting:** This matters a lot for agentic RAG specifically, since multi-step loops take several seconds - silent waiting feels broken even when the system is working correctly.

TRADEOFFS

Cost & Latency: Honest Tradeoff

Agentic RAG is not free - extra reasoning takes time and tokens

LLM CALLS PER QUERY COMPARISON



MITIGATION STRATEGIES

- **Tiered model usage:** Use a small/cheap model for routing, grading, and reflection nodes.
- **Reserve frontier models:** Reserve your best (most expensive) model only for final generation.
- **Aggressive caching:** Cache queries and decisions aggressively (Slide 21).
- **Hard retry caps:** Set strict retry caps so worst-case latency is bounded.

SECURITY RISK

Security: Prompt Injection

A risk unique to giving the LLM autonomy over retrieved content

⚠ The Hijack Threat

Retrieved documents (especially from the web) can contain text designed to hijack the LLM's instructions - e.g., "Ignore previous instructions and...". Tool-calling agents are especially vulnerable since a malicious doc could trigger unintended tool calls!

REQUIRED SECURITY MITIGATIONS

- **Untrusted data separation:** Treat all retrieved content as untrusted data, never as instructions - reinforce this in your system prompt.
- **Sanitize text:** Sanitize and strip suspicious patterns from retrieved text before it reaches the prompt.
- **Human-in-the-loop:** Restrict which tools are available for autonomous calling vs which require explicit user confirmation (especially tools with side effects like sending emails or writing to DB).

EVALUATION

Evaluating Agentic RAG

Standard RAG metrics still apply, but agentic systems need more

5 NEW METRICS FOR AGENTIC SYSTEMS

- **Routing accuracy:** Did it correctly decide RETRIEVE vs DIRECT vs CLARIFY?
- **Retry efficiency:** How often does it loop, and does that improve the answer?
- **Over-retrieval rate:** Calling retrieval when not needed (wasted cost).
- **Under-retrieval rate:** Skipping retrieval when needed (hallucination risk).
- **Tool selection accuracy:** Did it pick the right source in multi-tool setups?

Log the full graph path: Record every node visited and every decision made, not just the final answer. That is what lets you debug *why* a specific response went wrong.

OBSERVABILITY

Observability & Tracing

You cannot debug what you cannot see in a complex graph run

```
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_PROJECT"] = "agentic-rag-prod"
# Traces now automatically appear in your LangSmith dashboard
```

WHAT TO CAPTURE IN EVERY TRACE

- **Visual node execution:** Use LangSmith (or equivalent) to visualize every node execution, tool call, and state transition in a graph run.
- **Key metadata:** Capture which route was taken, query used, retrieved chunks, retry count, and grading/reflection verdicts.
- **2-minute inspection:** Turns "the agent gave a weird answer" from a guessing game into an instant trace inspection.

GO-LIVE CHECKLIST

Production Deployment Checklist

Essential requirements before shipping agentic RAG to production

- [x] Retry limits on every loop (grade to correct, reflect to generate)
- [x] Timeouts on every tool call
- [x] Fallback response for total failure ("I couldn't find an answer...")
- [x] Logging and tracing enabled for every graph run
- [x] Caching layer for repeated queries & decisions
- [x] Cheap model for routing/grading, strong model for generation
- [x] Guardrails against prompt injection from retrieved content
- [x] Evaluation suite running on a regular schedule

PITFALLS

Common Pitfalls

Mistakes that show up in almost every first agentic attempt

No retry cap
Causes infinite loops that silently rack up huge API costs.

Vague tool descriptions
The LLM calls the wrong tool or fails to call any tool at all.

Using your most expensive model for every node
Routing and grading do not need frontier-level reasoning.

Trusting retrieved content as instructions
Opens a severe security vulnerability to prompt injection.

Not logging intermediate steps
Makes it impossible to debug bad answers after the fact.

Going agentic when you didn't need to
Adds latency & cost with no real benefit for simple single-hop queries.

CHEAT SHEET

Patterns Quick Reference

Recap table of core architectures and when to use each

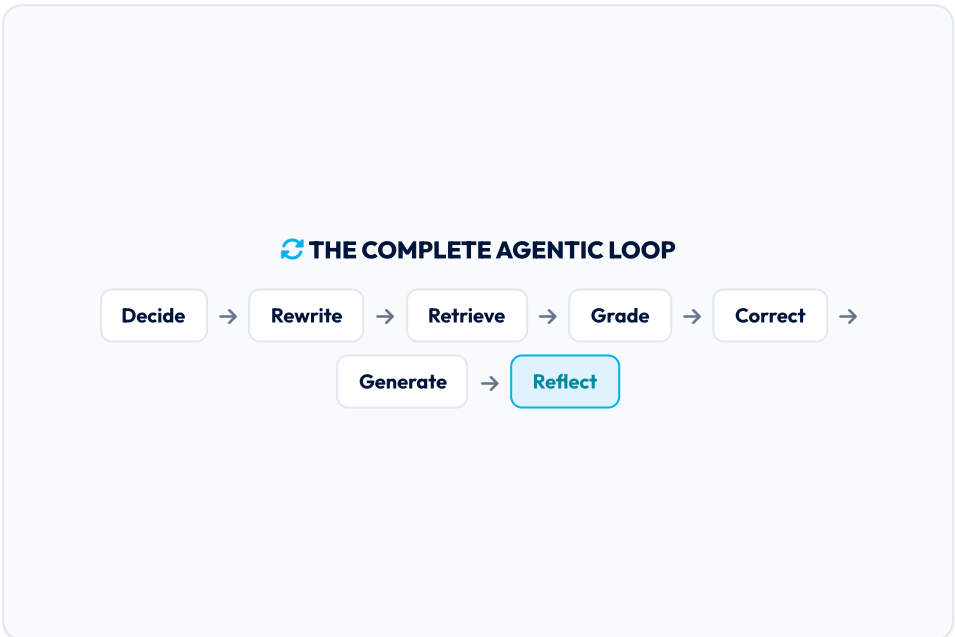
PATTERN	CORE IDEA	BEST FOR
Self-RAG	Model reflects on its own generated output	Reducing hallucination
Corrective RAG	Falls back to broader source on weak retrieval	Incomplete knowledge bases
Adaptive RAG	Routes dynamically by query complexity	Mixed simple/complex traffic
ReAct RAG	Reason to act to observe reasoning loop	General-purpose tool use
Multi-Agent RAG	Supervisor routing to specialized agents	Multiple distinct domains/sources

Pro Tip: You don't have to choose just one! As shown in Slide 16, state-of-the-art pipelines combine routing, grading, correction, and reflection into one unified LangGraph graph.

CONCLUSION

From Fixed Pipeline to a Reasoning System

Agentic RAG trades cost and latency for adaptability - the system decides when to look things up, what to search for, where from, and when it actually has enough context to answer confidently.



Save This Guide!
It is the reference guide for building anything beyond single-hop Q&A. Follow for more RAG + agent breakdowns.